

TOPIC – 2 BRUTE FORCE

1. Aim:

To write a program that sorts a list of numbers in ascending order.

To handle empty lists, single elements, identical elements, and negative numbers.

Algorithm:

1. Read the input list.
2. Check if the list is empty.
3. If not empty, apply sorting.
4. Use built-in sort () function.
5. Display the sorted list.

Code:

```
b1.py > ...
1  n = int(input("Enter number of elements: "))
2  arr = []
3
4  for i in range(n):
5      num = int(input("Enter element: "))
6      arr.append(num)
7
8  for i in range(n):
9      min_index = i
10     for j in range(i+1, n):
11         if arr[j] < arr[min_index]:
12             min_index = j
13     arr[i], arr[min_index] = arr[min_index], arr[i]
14
15  print("Sorted Array:", arr)
```

Input & output:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python b1.py
Enter number of elements: 4
Enter element: 2
Enter element: 5
Enter element: 4
Enter element: 6
Sorted Array: [2, 4, 5, 6]
```

Result: The list is successfully sorted for all given test cases.

2.Aim:

To sort an array using Selection Sort.

To understand sorted and unsorted partition method.

Algorithm:

1. Start from first element.
2. Find minimum in unsorted part.
3. Swap with current index.
4. Move boundary forward.
5. Repeat until sorted.

Code:

```
b2.py > ...
1  n = int(input("Enter number of elements: "))
2  arr = []
3
4  for i in range(n):
5      num = int(input("Enter element: "))
6      arr.append(num)
7
8  for i in range(n):
9      swapped = False
10     for j in range(0, n-i-1):
11         if arr[j] > arr[j+1]:
12             arr[j], arr[j+1] = arr[j+1], arr[j]
13             swapped = True
14     if not swapped:
15         break
16
17 print("Sorted Array:", arr)
```

Input & Output:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python b2.py
Enter number of elements: 4
Enter element: 76
Enter element: 45
Enter element: 32
Enter element: 55
Sorted Array: [32, 45, 55, 76]
```

Result: Selection Sort correctly arranges elements in ascending order

3.Aim:

To implement Bubble Sort with early stopping.

To reduce unnecessary comparisons.

Algorithm:

1. Traverse array.
2. Compare adjacent elements.
3. Swap if needed.
4. Track swap flag.
5. Stop if no swaps occur.

Code:

```
b3.py > ...
1  n = int(input("Enter number of elements: "))
2  arr = []
3
4  for i in range(n):
5      num = int(input("Enter element: "))
6      arr.append(num)
7
8  for i in range(1, n):
9      key = arr[i]
10     j = i - 1
11     while j >= 0 and arr[j] > key:
12         arr[j + 1] = arr[j]
13         j -= 1
14     arr[j + 1] = key
15
16  print("Sorted Array:", arr)
```

Input & Output:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python b3.py
Enter number of elements: 4
Enter element: 23
Enter element: 12
Enter element: 32
Enter element: 11
Sorted Array: [11, 12, 23, 32]
```

Result: The optimized Bubble Sort works correctly and stops early for sorted lists.

4.Aim:

To sort array using Insertion Sort.

To maintain stability when duplicates exist.

Algorithm:

1. Start from second element.
2. Compare with previous elements.
3. Shift larger elements.
4. Insert in correct position.
5. Repeat till end.

Code:

```
b4.py > ...
1  n = int(input("Enter number of elements: "))
2  arr = []
3
4  for i in range(n):
5      num = int(input("Enter element: "))
6      arr.append(num)
7
8  low = 0
9  high = n - 1
10
11 while low < high:
12     mid = (low + high) // 2
13     if arr[mid] > arr[mid + 1]:
14         high = mid
15     else:
16         low = mid + 1
17
18 print("Peak element index:", low)
19 print("Peak element value:", arr[low])
```

Input & Output:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python b4.py
Enter number of elements: 4
Enter element: 11
Enter element: 54
Enter element: 78
Enter element: 98
Peak element index: 3
Peak element value: 98
```

Result: Insertion Sort correctly sorts and preserves duplicate order.

5. Aim:

To find kth missing positive number.

To identify missing values from sorted array.

Algorithm:

1. Initialize missing count.
2. Traverse array.
3. Count gaps.
4. Stop at kth missing.
5. Return result.

Code:

```
b5.py > ...  
1 haystack = input("Enter main string: ")  
2 needle = input("Enter substring: ")  
3  
4 index = haystack.find(needle)  
5  
6 print("Index:", index)
```

Input & Output:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python b5.py  
Enter main string: apple  
Enter substring: pple  
Index: 1
```

Result: Program successfully finds kth missing positive number.

6. Aim:

To find a peak element in array.

To use binary search with $O(\log n)$ time.

Algorithm:

7. Apply binary search.
8. Find middle.
9. Compare neighbors.
10. Move accordingly.
11. Return peak index.

Code:

```
b6.py > ...
1  n = int(input("Enter number of elements: "))
2  arr = []
3
4  for i in range(n):
5      num = int(input("Enter element (sorted increasing): "))
6      arr.append(num)
7
8  k = int(input("Enter k value: "))
9
10 missing_count = 0
11 current = 1
12 i = 0
13
14 while True:
15     if i < n and arr[i] == current:
16         i += 1
17     else:
18         missing_count += 1
19         if missing_count == k:
20             print("Kth Missing Number:", current)
21             break
```

Input & Output:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python b6.py
Enter number of elements: 4
Enter element (sorted increasing): 23
Enter element (sorted increasing): 32
Enter element (sorted increasing): 45
Enter element (sorted increasing): 67
Enter k value: 3
Kth Missing Number: 3
```

Result: Peak element found successfully in logarithmic time.

8. Aim:

To find words that are substring of another word.

To return all such words.

Algorithm:

1. Compare each word with others.
2. Check substring condition.
3. Add matching words.
4. Avoid duplicates.
5. Return result list.

Code:

```
b8.py > ...
1  import math
2
3  n = int(input("Enter number of points: "))
4  points = []
5
6  for i in range(n):
7      x = float(input("Enter x coordinate: "))
8      y = float(input("Enter y coordinate: "))
9      points.append((x, y))
10
11  min_dist = float('inf')
12  pair = ()
13
14  for i in range(n):
15      for j in range(i+1, n):
16          dist = math.sqrt((points[i][0]-points[j][0])**2 + (points[i][1]-points[j][1])**2)
17          if dist < min_dist:
18              min_dist = dist
19              pair = (points[i], points[j])
20
21  print("Closest Pair:", pair)
```

Input & Output:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python b8.py
Enter number of points: 3
Enter x coordinate: 1
Enter y coordinate: 2
Enter x coordinate: 1
Enter y coordinate: 3
Enter x coordinate: 1
Enter y coordinate: 4
Closest Pair: ((1.0, 2.0), (1.0, 3.0))
Minimum Distance: 1.0
```

Result: To find words that are substring of another word.

To return all such words.

9. Aim:

To find the closest pair of points in a 2D plane using brute force.

To compute the minimum Euclidean distance between all pairs.

Algorithm:

1. Take list of points as input.
2. Initialize minimum distance as infinity.
3. Compare every pair of points.
4. Compute Euclidean distance.
5. Store pair with smallest distance.

Code:

```
b9.py > ...
7   for i in range(n):
8       x = float(input("Enter x coordinate: "))
9       y = float(input("Enter y coordinate: "))
10      points.append((x, y))
11
12      hull = []
13
14      for i in range(n):
15          for j in range(n):
16              if i != j:
17                  left = right = 0
18                  for k in range(n):
19                      if k != i and k != j:
20                          val = orientation(points[i], points[j], points[k])
21                          if val > 0:
22                              left += 1
23                          elif val < 0:
24                              right += 1
25                  if left == 0 or right == 0:
26                      hull.append(points[i])
27                      hull.append(points[j])
```

Input & Output:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python b9.py
Enter number of points: 2
Enter x coordinate: 1
Enter y coordinate: 2
Enter x coordinate: 1
Enter y coordinate: 3
Convex Hull Points: [(1.0, 2.0), (1.0, 3.0)]
```

Result: The program successfully finds the closest pair using brute force.

Time Complexity: $O(n^2)$

10. Aim:

To find the convex hull of given points using brute force.

To determine boundary points forming outer polygon.

Algorithm:

1. Take all pairs of points.
2. Form line between them.
3. Check if all other points lie on one side.
4. If yes, include edge in hull.
5. Return hull points.

Code:

```
b10.py > ...
5   return math.sqrt((c1[0]-c2[0])**2 + (c1[1]-c2[1])**2)
6
7   n = int(input("Enter number of cities: "))
8   cities = []
9
10  for i in range(n):
11      x = float(input("Enter x coordinate: "))
12      y = float(input("Enter y coordinate: "))
13      cities.append((x, y))
14
15  min_path = None
16  min_cost = float('inf')
17
18  for perm in itertools.permutations(cities[1:]):
19      path = [cities[0]] + list(perm) + [cities[0]]
20      cost = 0
21      for i in range(len(path)-1):
22          cost += distance(path[i], path[i+1])
23      if cost < min_cost:
24          min_cost = cost
25          min_path = path
```

Input & Output:

```
PS C:\Users\Anandpaul\OneDrive\Desktop\DAA_Lab> python b10.py
Enter number of cities: 2
Enter x coordinate: 2
Enter y coordinate: 3
Enter x coordinate: 2
Enter y coordinate: 3
Minimum Path: [(2.0, 3.0), (2.0, 3.0), (2.0, 3.0)]
Minimum Cost: 0.0
```

Result: Convex hull boundary points are successfully identified.

11. Aim:

To compute convex hull of 2D points using brute force.

To return hull points in counter-clockwise order.

Algorithm:

1. Consider all pairs of points.
2. Form line equations.
3. Check orientation of remaining points.
4. Select boundary edges.
5. Arrange points counter-clockwise.

Code:

Input & Output:

Result: Convex hull points are correctly determined.

12. Aim:

To solve TSP using exhaustive search method.

To find minimum cost Hamiltonian cycle.

Algorithm:

1. Define distance function.
2. Fix starting city.
3. Generate permutations of remaining cities.
4. Calculate total path cost.
5. Return minimum cost path.

Code:

Input & Output:

Result: TSP solved using exhaustive search.

Time Complexity: $O(n!)$

13. Aim:

To solve assignment problem using permutation method.

To minimize total cost of assigning workers to tasks.

Algorithm:

1. Take cost matrix.
2. Generate all permutations of assignments.
3. Calculate total cost.
4. Compare all costs.
5. Return minimum cost assignment.

Code:

Input & Output:

Result: Optimal worker-task assignment is found successfully.

Time Complexity: $O(n!)$

14. Aim:

Optimal worker-task assignment is found successfully.

Time Complexity: $O(n!)$

Algorithm:

1. Generate all subsets of items.
2. Check feasibility using weight constraint.
3. Calculate total value.
4. Track maximum value.
5. Return optimal subset.

Code:**Input & Output:**

Result: Knapsack solution is correctly obtained using exhaustive search.

Time Complexity: $O(2^n)$

